Лабораторная работа №4: Рефакторинг спагетти-кода: устранение нарушений SRP и OCP

Курс: Объектно-ориентированное программирование 1 (CS2203)
Университет: КазНУ им. аль-Фараби
Преподаватель: Санжар Аубакиров, PhD
Неделя: 4 из 10
Формат: Лабораторная работа, 3 академических часа (135 минут)
Среда выполнения: Python 3.12+, pytest, Git, VS Code / PyCharm

---

1. Цель и контекст работы

Цель работы: Получить практический навык выявления запахов кода (**Code Smells**) в легаси-системе и провести последовательный безопасный рефакторинг монолитного класса («God Object») на слабосвязанные модули в строгом соответствии с принципами **Single Responsibility Principle (SRP)** и **Open/Closed Principle (OCP)**.
Студентам предоставляется реальный спагетти-класс бортового валидатора SpaghettiValidator, в котором чтение физических датчиков, финансовые расчеты, запись в SQLite-файл, воспроизведение звуков и отправка телеметрии свалены в один 200-строчный метод.
Задача — разобрать этот монолит на изолированные компоненты с высокой связностью и закрыть ядро терминала от модификаций при добавлении новых платежных методов.

---

2. Структура репозитория и требования к сдаче

Работа выполняется в папке lab04_solid_refactoring/:

```
lab04_solid_refactoring/
├── legacy/
│   ├── __init__.py
│   └── spaghetti_terminal.py  # Исходный монолитный код (для сравнительных тестов)
├── refactored/
│   ├── __init__.py
│   ├── hardware.py           # Абстракции и драйверы железа (ридер, пищалка, дисплей)
│   ├── repository.py         # Сохранение транзакций (хранилище)
│   ├── pricing.py            # Сервис расчета тарифа
│   ├── payment_methods.py    # Полиморфные стратегии оплаты (OCP)
│   └── terminal.py           # Чистый терминал-оркестратор (SRP)
├── tests/
│   ├── __init__.py
│   ├── test_equivalence.py   # Тесты эквивалентности поведения (Golden Master)
│   └── test_refactored.py    # Модульные тесты отрефакторенных классов
└── README.md                 # Матрица рефакторинга: Code Smells до и после
```

Критерии оценки (20 баллов):

Блок	Задание	Баллы
Блок 1	Архитектурный аудит монолита: документирование запахов кода в README.md	4
Блок 2	Рефакторинг монолита с выделением 4+ независимых классов по SRP	8
Блок 3	Обеспечение OCP: добавление нового метода оплаты без изменения terminal.py	4
Блок 4	Тестирование эквивалентности поведения до и после рефакторинга	4

---

3. Практические задания

Блок 1. Архитектурный аудит легаси-монолита (4 балла)

В legacy/spaghetti_terminal.py проанализируйте следующий монолитный метод:

```
class SpaghettiValidator:
    def tap_and_pay(self, raw_rfid_bytes: bytes, payment_mode: str, passenger_category: str):
        # 1. Парсинг байт ридера
        # 2. Вычисление скидок и тарифа через вложенные if-else
        # 3. Прямой SQL-запрос INSERT INTO transactions...
        # 4. Вызов системного звукового бипера winsound / os.system
        # 5. Отрисовка сообщения на ЖК-экране
        # 6. Отправка аварийного алерта при сбое
```

В файле README.md заполните таблицу: - Имя запаха кода (*God Object*, *Divergent Change*, *Shotgun Surgery*, *Primitive Obsession*). - В каких строках исходного файла он проявляется. - Какой принцип SOLID нарушен.

Блок 2. Пошаговое разделение по принципу SRP (8 баллов)

В папке refactored/ проведите декомпозицию:

1. **hardware.py**:
    - CardReaderProtocol и его эмулятор NfcCardReader.
    - BuzzerProtocol и ScreenProtocol для обратной связи пассажиру (зеленый/красный сигнал, звуковой сигнал beep_success() / beep_error()).
2. **pricing.py**:
    - FareCalculationService — класс, единственный повод для изменения которого — изменение тарифной сетки города.
3. **repository.py**:

- TransactionRepositoryProtocol и реализация InMemoryTransactionRepository / SqliteTransactionRepository. Единственный повод для изменения — смена схемы БД или способа персистентности.

4. **terminal.py**:
   - TransitTerminal — класс, который только оркестрирует последовательность действий (прочитать карту → рассчитать тариф → списать средства → зафиксировать транзакцию → выдать звуковой и визуальный сигнал).

Блок 3. Обеспечение принципа OCP (4 балла)

В файле refactored/payment_methods.py: - Создайте протокол PaymentStrategy: python class PaymentStrategy(Protocol): def process(self, amount: Decimal) -> PaymentResult: ... - Реализуйте стратегии: - TransitCardPaymentStrategy - KaspiQrPaymentStrategy - BankContactlessPaymentStrategy
Критерий приемки: Добавление нового способа BluetoothPaymentStrategy не должно требовать изменения ни одной строчки в terminal.py!

Блок 4. Тестирование эквивалентности поведения (pytest) (4 балла)

В tests/test_equivalence.py: 1. Напишите тест «Golden Master»: на фиксированном наборе входных данных (10 транзакций с разными картами, балансами и тарифами) отрефакторенная система выдает в точности такие же финансовые результаты и статусы, что и легаси-спагетти код. 2. Проверьте, что отрефакторенный TransitTerminal легко тестируется с помощью фиктивных мок-объектов без необходимости открывать реальные соединения с БД или файлами.
pytest tests/ -v

---

4. Контрольные вопросы для защиты работы

1. Что такое «Divergent Change» (разнородные изменения) и «Shotgun Surgery» (стрельба дробью)? Как SRP помогает от них избавиться?
2. Почему прямое внедрение конкретного класса репозитория в терминал нарушает принцип Low Coupling?
3. В чем опасность проведения рефакторинга без наличия автоматических тестов?
4. Как с помощью принципа OCP мы защитили кодовую базу от регрессионных ошибок при внедрении оплаты через QR?